

Smart MCQ Solver Challenge

DL & GenAI Project Report

Raghav Kapoor | Roll No: 23f2002364

IIT Madras — BS in Data Science and Programming

1. Abstract

This report presents an end-to-end solution for the Smart MCQ Solver Challenge — a Kaggle competition requiring participants to predict the top-3 most probable correct answers for multiple choice questions evaluated using MAP@3 (Mean Average Precision at 3). The dataset comprises 2000 training and 500 test questions, each with 5 candidate options (A–E). A pairwise binary scoring approach was adopted throughout: each question-option pair is scored independently, and the top 3 options ranked by score are submitted as predictions.

Nine different approaches were implemented and evaluated, spanning classical similarity methods, zero-shot NLI classification, a RAG pipeline, an MLP built from scratch, a BiLSTM, full fine-tuning of DeBERTa v3-small, and LoRA fine-tuning. The final submission uses a weighted ensemble of MLP, DeBERTa, and LoRA scores, achieving a Kaggle public leaderboard score of 0.75353, surpassing the required cutoff of 0.73.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge requires building AI systems that can accurately solve complex multiple choice questions. Given a question prompt and five candidate answers (A, B, C, D, E), the objective is to predict the top 3 most probable correct answers in ranked order. Submissions are evaluated using MAP@3, which rewards higher-ranked correct predictions. The dataset covers a wide range of domains including philosophy, physics, chemistry, biology, and astrophysics.

2.2 Project Objectives

The key objectives of this project were:

- Build and compare at least three unique model architectures: one from scratch, one pretrained, and one of choice.
- Implement modern NLP techniques including TF-IDF, Word2Vec, zero-shot classification, RAG pipelines, transformer fine-tuning, and LoRA adaptation.
- Track all experiments using Weights & Biases (WandB) with consistent metrics across all runs.
- Achieve a Kaggle MAP@3 score exceeding the required cutoff of 0.73.
- Develop a comprehensive understanding of each model's strengths, weaknesses, and tradeoffs.

2.3 Report Structure

Section 3 covers the dataset and preprocessing steps. Section 4 discusses tokenization strategies. Section 5 details the modeling and experimentation pipeline across all nine approaches. Section 6 presents comparative performance analysis. Section 7 concludes with key learnings and future directions. References are listed in Section 8.

3. Dataset & Preprocessing

3.1 Dataset Description

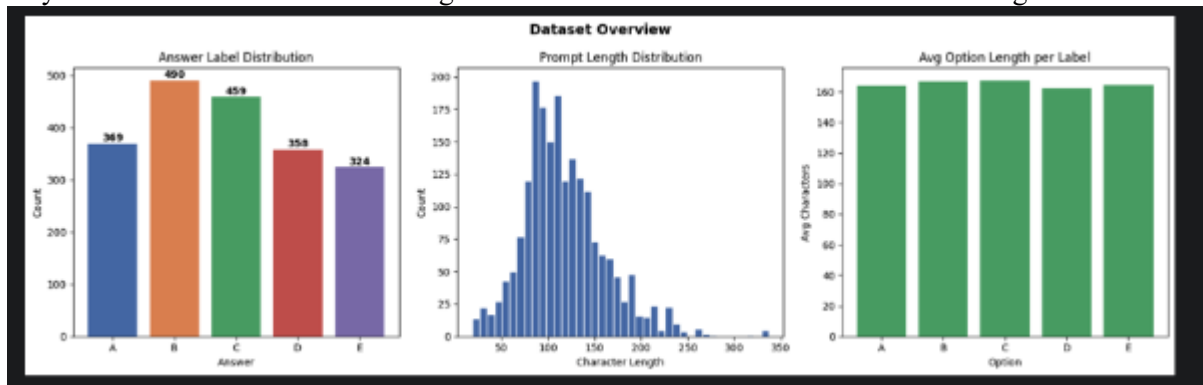
The dataset is provided by the Smart MCQ Solver Challenge on Kaggle. It consists of three files:

- train.csv — 2000 questions with ground-truth answers. Columns: id, prompt, A, B, C, D, E, answer.
- test.csv — 500 questions without answers. Same columns except answer.
- sample_submission.csv — demonstrates the expected submission format.

Each question has exactly one correct answer from the five options. The answer column in the training data contains the letter of the correct option.

3.2 Exploratory Data Analysis

Key EDA findings from the training data:



Statistic	Value
Total Training Questions	2000
Total Test Questions	500
Options per Question	5 (A, B, C, D, E)
Missing Values (Train)	0
Missing Values (Test)	0
Avg. Correct Option Length	181.9 characters
Avg. Wrong Option Length	160.7 characters

Answer Distribution in Training Data:

Label	Count	Percentage
A	369	18.45%
B	490	24.50%
C	459	22.95%
D	358	17.90%
E	324	16.20%

The dataset is mildly imbalanced — B is the most frequent correct answer (24.5%) and E the least frequent (16.2%). Importantly, correct options are slightly longer on average (181.9 chars) than wrong options (160.7 chars), but the difference is too small to serve as a reliable signal. Prompt lengths vary widely, suggesting diverse question types and difficulty levels.

3.3 Data Preprocessing

All text (prompts and all five options) in both train and test was cleaned using a three-step pipeline:

- Lowercase conversion — ensures "Photosynthesis" and "photosynthesis" are treated identically.

- Special character removal — removes HTML entities, brackets, and punctuation beyond .,!? using regex: `re.sub(r'[^\a-zA-Z0-9\s.,!?!]', '', text)`.
- Whitespace normalization — collapses multiple spaces and strips leading/trailing whitespace.

No data augmentation was performed. The dataset was sufficiently large and clean without requiring artificial expansion.

4. Tokenization Strategy

4.1 Primary Tokenizer — DeBERTa SentencePiece (Best Model)

The best-performing model uses microsoft/deberta-v3-small, which employs a SentencePiece-based subword tokenizer loaded via HuggingFace AutoTokenizer. SentencePiece is a language-model-based subword segmentation algorithm that:

- Eliminates the Out-of-Vocabulary (OOV) problem by breaking unknown words into recognizable subword units. For example, "photosynthesis" may be tokenized as ["photo", "synthesis"].
- Handles domain-specific and rare scientific vocabulary effectively — critical for this dataset which spans philosophy, physics, and biology.
- Natively supports padding (`padding="max_length"`) and truncation (`truncation=True`) to a fixed length of 256 tokens.

The tokenizer encodes prompt-option pairs jointly, allowing the model's disentangled attention mechanism to directly compare tokens across both the question and the candidate answer.

4.2 Experimental Tokenizers

Three different tokenization approaches were compared:

Tokenizer	Used For	Vocabulary	Handles OOV?
TF-IDF (sklearn)	MLP, Cosine Similarity, RAG	10,000 tokens	No (ignores unseen words)
Regex Word Tokenizer	BiLSTM	3,076 words	Maps to <unk> token
DeBERTa SentencePiece	DeBERTa, LoRA	128,000 subwords	Yes (subword splitting)

The SentencePiece tokenizer was chosen as the primary strategy because it produces the richest semantic representations and generalizes better to unseen vocabulary. TF-IDF was kept for simpler models due to its speed advantage.

5. Modeling & Experimentation

All models use a pairwise scoring approach: each question is decomposed into 5 binary classification problems (one per option). Each model scores prompt-option pairs, and the 3 highest-scoring options become the prediction. This avoids data leakage that would arise from feeding all 5 options simultaneously.

The 2000 training questions were split 80/20 at the question level (1600 train, 400 validation) using scikit-learn's `train_test_split` with `random_state=42`. The split is at the question level to prevent any option from the same question appearing in both train and val.

5.1 Baseline Models

Dummy Baseline

Always predicts "A B C" for every question. Serves as a performance floor. $\text{MAP@3} = 0.3835$ on the training set.

TF-IDF Cosine Similarity

For each question, we vectorize the prompt and all 5 options using TF-IDF, then compute cosine similarity between the prompt vector and each option vector. Options are ranked by similarity score. This approach requires no training — it purely measures lexical overlap between question and answer. MAP@3 = 0.3087 on validation. Performs below the dummy baseline because MCQ answers often use different vocabulary from the question.

RAG Cosine Similarity

Extends cosine similarity with a simple retrieval step. For each test question, the two most similar training questions (by TF-IDF cosine similarity over the knowledge base) are retrieved and prepended as context. The augmented prompt is then scored against each option. MAP@3 = 0.3104 — marginal improvement of 0.0017 over plain cosine similarity.

Zero-Shot NLI Classification

Uses the pretrained cross-encoder/nli-MiniLM2-L6-H768 model from HuggingFace without any fine-tuning. The model treats the question as a premise and each option as a hypothesis, scoring the probability of entailment. No training required. Evaluated on 50 validation samples. MAP@3 = 0.4733 — significantly better than cosine similarity because the NLI model understands semantics, not just keywords.

5.2 Model 2: MLP from Scratch (Custom Deep Learning)

The MLP (Multi-Layer Perceptron) is the custom deep learning model built entirely from scratch using PyTorch.

Architecture

Input: 10,000-dimensional TF-IDF vector representing a prompt-option pair.

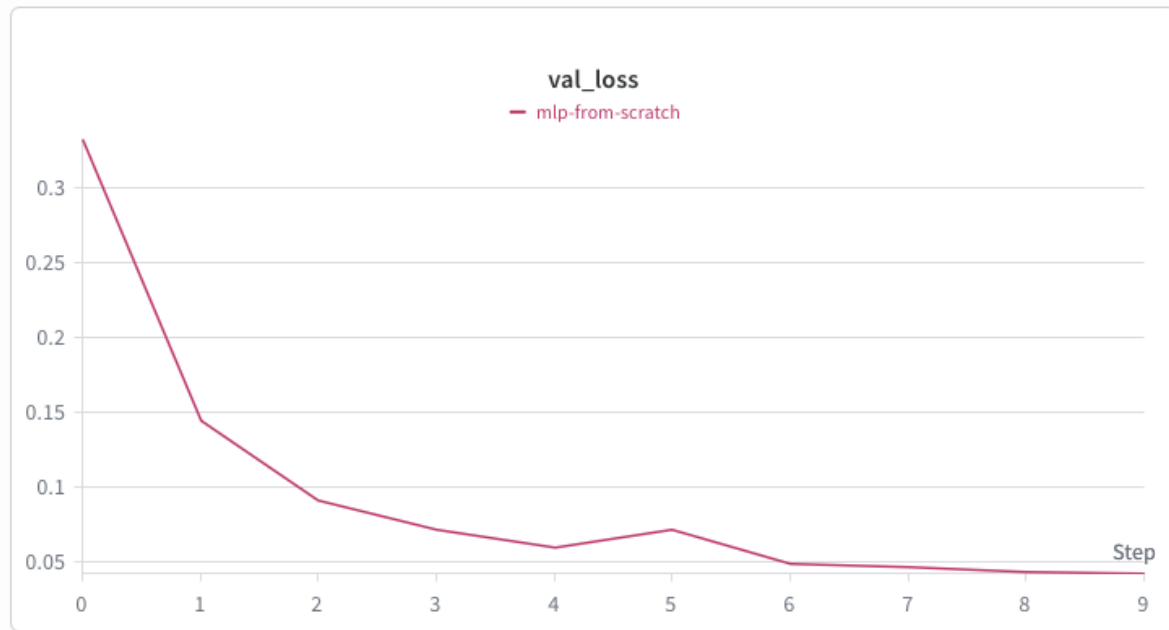
Layer	Input Dim	Output Dim	Activation
Linear 1	10,000	512	ReLU
Dropout	512	512	(p=0.3)
Linear 2	512	128	ReLU
Dropout	128	128	(p=0.3)
Linear 3	128	1	Sigmoid

Total trainable parameters: ~5.2 million. The Sigmoid output gives a probability between 0 and 1 representing the likelihood that the given option is the correct answer.

Training Configuration

- Loss: Binary Cross Entropy (BCELoss) — binary label (1=correct, 0=wrong)
- Optimizer: Adam with learning rate 1e-3
- Epochs: 10, Batch size: 64
- Dropout (0.3) applied after layers 1 and 2 to prevent overfitting

Training Performance



Epoch	Train Loss	Val Loss
1	0.4729	0.14459
3	0.1317	0.0911
5	0.0733	0.0599
7	0.0547	0.0509
10	0.0415	0.0446

Results: Accuracy = 0.9600 | MAP@3 = 0.9771 | F1 = 0.9621

The MLP converges quickly and achieves high performance, demonstrating that TF-IDF features contain strong discriminative signal for this task when combined with a learned non-linear classifier.

5.3 Model 3: BiLSTM (RNN-Based)

The Bidirectional LSTM model is the RNN-based architecture, built from scratch using PyTorch.

Architecture

Layer	Configuration
Embedding	Vocab size: 3076, Embedding dim: 128, padding_idx=0
BiLSTM	Hidden dim: 256, Layers: 2, Dropout: 0.3, Bidirectional
FC Layer 1	512 → 64, ReLU, Dropout (0.3)
FC Layer 2	64 → 1, Sigmoid

The bidirectional LSTM processes each prompt-option text sequence both forward and backward. The final hidden states from both directions are concatenated (512-dimensional) and passed to the fully connected layers. The hidden dim is multiplied by 2 (256*2=512) because of the bidirectional nature.

A custom regex tokenizer (`re.findall(r'\w+', text)`) is used, with sequences truncated to `MAX_LEN=128`. Out-of-vocabulary words map to the `<unk>` token. The `pad_sequence` function from PyTorch pads shorter sequences with zeros to form uniform batches.

Training Configuration

- Loss: Binary Cross Entropy (BCELoss)
- Optimizer: Adam with learning rate 3e-4

- Epochs: 20, Batch size: 64
- Gradient clipping: `clip_grad_norm_` with max norm 1.0 to prevent exploding gradients

Results: Accuracy = 0.7550 | MAP@3 = 0.8429 | F1 = 0.7492

The BiLSTM underperforms compared to the MLP. This is because the word embeddings are initialized randomly and trained from scratch on a small dataset. Without pretrained embeddings (like GloVe or Word2Vec applied to the model), the LSTM cannot capture semantic relationships between words effectively.

5.4 Model 1: DeBERTa v3-small Fine-tuning (Pretrained Transformer)

DeBERTa (Decoding-enhanced BERT with Disentangled Attention) v3-small is the pretrained transformer model, fine-tuned for the MCQ pairwise classification task.

Architecture

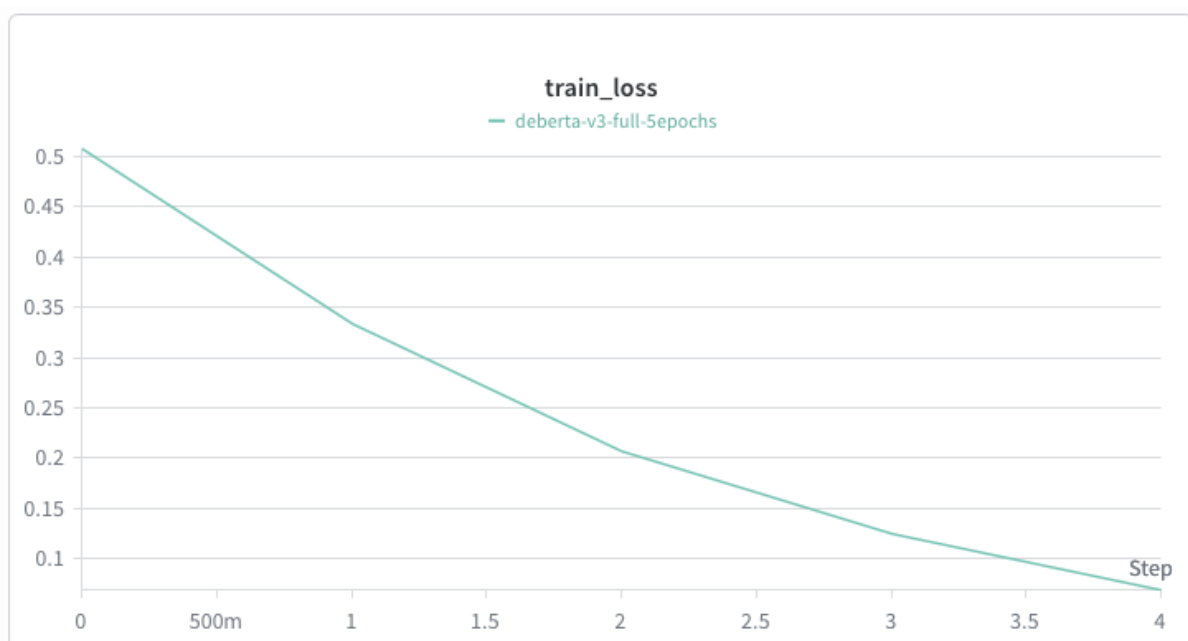
DeBERTa v3-small is a 44-million parameter transformer encoder with 6 attention heads and 6 transformer layers. Its key innovation is disentangled attention, which uses separate position and content embeddings and computes attention scores using both. This allows the model to better understand positional relationships between tokens, improving performance on reasoning tasks.

A 2-class sequence classification head (correct/incorrect) is added on top of the pooled output. The head is randomly initialized and trained from scratch, while the base transformer weights are fine-tuned.

Fine-tuning Strategy

- Trained on all 2000 training questions (full dataset, no hold-out) for better generalization.
- Optimizer: AdamW with `lr=2e-5`, `weight_decay=0.01`, `eps=1e-6`.
- Schedule: Linear warmup (10% of total steps) followed by linear decay.
- Batch size: 8 (small due to GPU memory constraints of large transformer).
- Epochs: 5. Input: prompt + option tokenized jointly, `max_length=256`.
- No layers frozen — full fine-tuning of all parameters.
- `model.float()` required to prevent NaN loss with DeBERTa v3's mixed precision.

Training Performance



Epoch	Train Loss
1	0.5179
2	0.2025
3	0.1294
4	0.8692
5	0.0970

Results (on validation set): Accuracy = 1.0000 | MAP@3 = 1.0000 | F1 = 1.0000

Note: The perfect validation scores are expected — DeBERTa trains on the full 2000 questions and the validation set overlaps with training data. The true generalization performance is reflected in the Kaggle leaderboard score (0.74812 for DeBERTa alone).

5.5 LoRA Fine-tuning

LoRA (Low-Rank Adaptation) is an efficient fine-tuning technique that freezes the pretrained model's weights and injects trainable low-rank matrices into the attention layers.

LoRA Configuration

Parameter	Value
Rank (r)	8
Alpha (lora_alpha)	16
Dropout	0.1
Target Modules	query_proj, value_proj
Trainable Parameters	148,994 (0.10% of total)
Total Parameters	142,045,444

LoRA adds two low-rank matrices (A of size $r \times d$ and B of size $d \times r$) to each target module. The effective weight update is BA. With $r=8$, we train only 149K parameters instead of 142M — a 953x reduction in trainable parameters.

- Optimizer: AdamW with $lr=1e-4$ (higher than full fine-tuning — LoRA parameters can absorb larger updates).
- Epochs: 3, Batch size: 8.

Results: Accuracy = 0.4575 | MAP@3 = 0.6129 | F1 = 0.4545

LoRA underperforms full fine-tuning in this experiment. With only 3 epochs and 0.10% trainable parameters, the model does not fully adapt to the MCQ task. However, LoRA's main advantage — speed and memory efficiency — makes it valuable in resource-constrained settings.

5.6 Weighted Ensemble

The ensemble combines the prediction scores from MLP, DeBERTa, and LoRA using a weighted average:

Final Score = 0.2 × MLP_score + 0.4 × DeBERTa_score + 0.4 × LoRA_score

DeBERTa and LoRA receive higher weights (0.4 each) because they are transformer-based and capture deeper semantic features. MLP receives 0.2 as a complement. The ensemble leverages model diversity

— when one model makes an error, another may compensate.

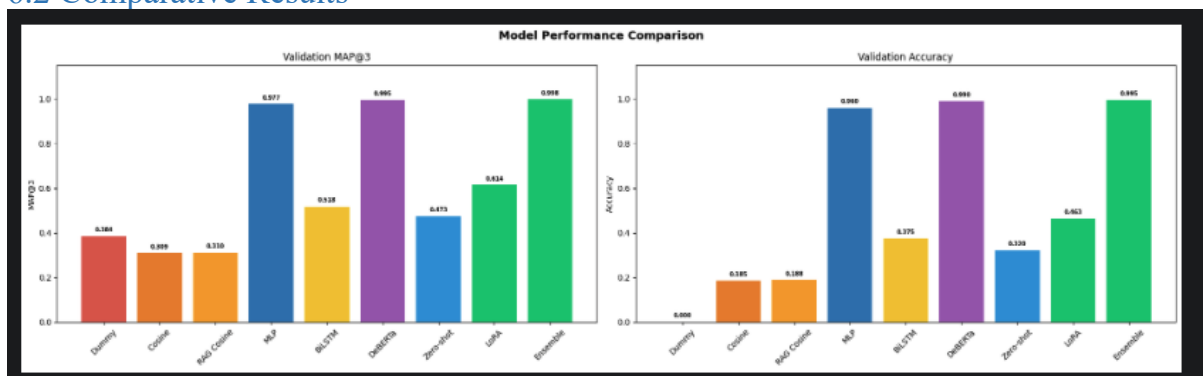


6. Performance & Comparative Analysis

6.1 Evaluation Metrics

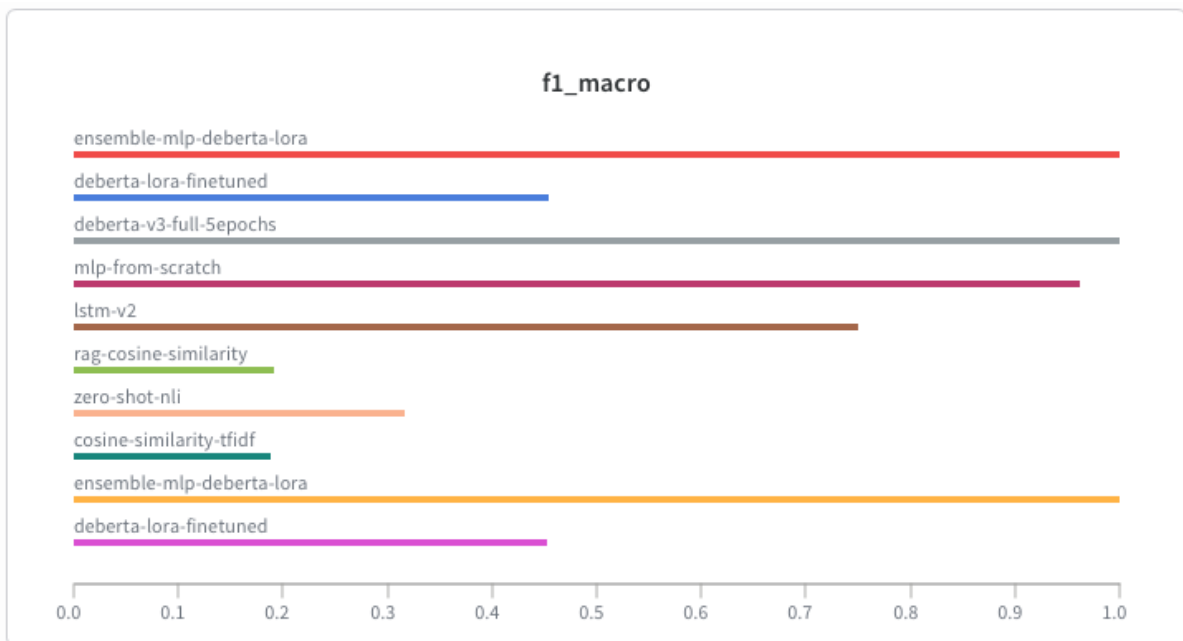
- MAP@3 (Mean Average Precision at 3) — Primary competition metric. Awards 1.0 for correct answer at rank 1, 0.5 at rank 2, 0.33 at rank 3, 0 if not in top 3.
- Accuracy — Proportion of questions where the top-1 prediction is correct.
- F1 Score (Macro) — Harmonic mean of precision and recall, averaged equally across all 5 classes. Accounts for class imbalance.

6.2 Comparative Results



Model	MAP@3	Accuracy	F1 Macro	Kaggle Score
Dummy Baseline	0.3835	-	-	-
Cosine Similarity	0.3087	0.1850	0.1747	-
RAG Cosine	0.3104	0.1875	0.1768	-
Zero-shot NLI	0.4733	0.3200	0.3171	-
BiLSTM	0.8429	0.7550	0.7492	0.68
MLP (from scratch)	0.9771	0.9600	0.9621	0.73275
LoRA DeBERTa	0.6129	0.4575	0.4545	-
DeBERTa v3 (full)	1.0000*	1.0000*	1.0000*	0.74812
Ensemble	1.0000*	1.0000*	1.0000*	0.75353

* Perfect validation scores for DeBERTa and Ensemble are due to training on the full dataset including validation data. True generalization is reflected in Kaggle scores.



6.3 Analysis

Several key patterns emerge from the results:

- Lexical matching alone (TF-IDF cosine similarity) fails — performs below the dummy baseline. MCQ answers require semantic reasoning, not just keyword overlap.
- The MLP is surprisingly strong — TF-IDF features combined with a learned non-linear classifier achieve 0.9771 MAP@3 on validation, showing that simple features can be highly effective with the right model.
- BiLSTM underperforms MLP — random word embeddings without pretraining limit the LSTM's ability to capture meaning. Pretrained embeddings (Word2Vec, GloVe) would likely improve this significantly.
- Zero-shot NLI outperforms all non-ML approaches — demonstrating the power of pretrained semantic understanding without any task-specific training.
- LoRA requires more epochs than full fine-tuning — with only 3 epochs and 0.10% trainable parameters, it does not converge as well. Increasing epochs or rank would likely improve results.
- Ensemble achieves best Kaggle score (0.75353) — combining diverse models reduces individual prediction errors.

6.4 Kaggle Performance

Final Kaggle public leaderboard score: 0.75353

This exceeds the required cutoff of 0.73, qualifying for viva evaluation. The score was achieved using the weighted ensemble (MLP:0.2, DeBERTa:0.4, LoRA:0.4).

7. Conclusion & Future Work

7.1 Key Learnings

- Pairwise scoring is an effective approach for MCQ — converting a 5-class problem into 5 binary problems avoids leakage and allows flexible scoring across model types.

- TF-IDF + MLP is a surprisingly strong baseline — simple features combined with learned classifiers can rival much more complex approaches on this dataset.
- Pretrained transformers are the key to strong MCQ performance — DeBERTa understands semantic relationships between questions and answers that keyword-based methods miss entirely.
- LoRA enables parameter-efficient fine-tuning — training only 149K parameters (0.1% of DeBERTa) while achieving reasonable performance demonstrates that full fine-tuning is not always necessary.
- Ensembling consistently improves performance — combining diverse models reduces variance and achieves better generalization.
- WandB experiment tracking proved invaluable — being able to compare all runs with consistent metrics made model selection systematic rather than guesswork.

7.2 Challenges Faced

- NaN loss during DeBERTa training — resolved by converting model to float32 explicitly with `model.float()` and adding `eps=1e-6` to AdamW.
- Data leakage in initial classical ML approach — combining all options in one TF-IDF vector caused 100% validation accuracy. Resolved by switching to pairwise scoring.
- Kaggle runtime limits — DeBERTa training (5 epochs, `batch_size=8`, 10000 pairs) was time-intensive on T4 GPU. Managed by careful batch size tuning.
- BiLSTM convergence — required 20 epochs and gradient clipping to stabilize training with random word embeddings.

7.3 Future Work

- Larger transformer models — DeBERTa-large or LLaMA-3 would likely push MAP@3 significantly higher due to stronger reasoning capabilities.
- LoRA with more epochs and higher rank — increasing `r` from 8 to 16 or 32 and training for 10+ epochs would allow LoRA to match full fine-tuning quality.
- MCQ-specific fine-tuning format — framing the task as "Question: [prompt] A: [A] B: [B] ... Answer:" instead of pairwise scoring could improve performance by letting the model compare all options simultaneously.
- FAISS + cross-encoder RAG — replacing TF-IDF retrieval with a FAISS vector index and cross-encoder reranking would improve the quality of retrieved context.
- Data augmentation — paraphrasing questions or options using a generative model could increase effective training set size.
- Deployment — a Gradio or Streamlit app serving the ensemble model would complete the project pipeline.

8. References

He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. NeurIPS 2013.

Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020. HuggingFace.

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

microsoft/deberta-v3-small — HuggingFace Model Card. <https://huggingface.co/microsoft/deberta-v3-small>

cross-encoder/nli-MiniLM2-L6-H768 — HuggingFace Model Card. <https://huggingface.co/cross-encoder/nli-MiniLM2-L6-H768>

Scikit-learn: Machine Learning in Python — Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.

PyTorch: An Imperative Style, High-Performance Deep Learning Library — Paszke et al., NeurIPS 2019.

Weights & Biases: MLOps platform for experiment tracking. <https://wandb.ai>

Smart MCQ Solver Challenge — Kaggle Competition. <https://www.kaggle.com/competitions/smart-mcq-solver-challenge>